

Actividad uno segundo corte

Cleyver Quiroz Martinez

Ejercicio: Base de datos de Tienda en Línea

Profesor:

Cristian Cuadrado

Ingeniero de sistemas

UNIVERSIDAD DEL SINU

Elias Bechara Zainum

Seccional - Cartagena

2024

Contenido

Objetivo de la actividad.....	4
Objetivo #1:	4
Objetivo #2:	4
Explicación para tener en cuenta	4
Diseño de la base de datos.....	5
Creación de tablas y relaciones en SQL	5
Relaciones entre Tablas.....	7
Inserción de datos	8
Consultas SQL	9
Consultas Básicas	9
Consultas Avanzadas	9
Consultas de Actualización.....	10
Consultas de Eliminación.....	10
Justificación del diseño.....	11
Normalización y Eficiencia.....	11
• Productos:	11
• Clientes:	11
• Pedidos.....	11
• Detalles_Pedido	11
Relaciones Claras y Consistentes	12

Flexibilidad y Escalabilidad 12

Consultas Eficientes..... 12

Integridad de los Datos 12

Objetivo de la actividad

Objetivo #1:

Los estudiantes diseñarán y crearán una base de datos relacional para gestionar una tienda en línea, que incluya información de productos, clientes y pedidos. También realizarán consultas para extraer datos útiles.

Objetivo #2:

Probar que la base de datos sea funcional por medio de un MVC básico.

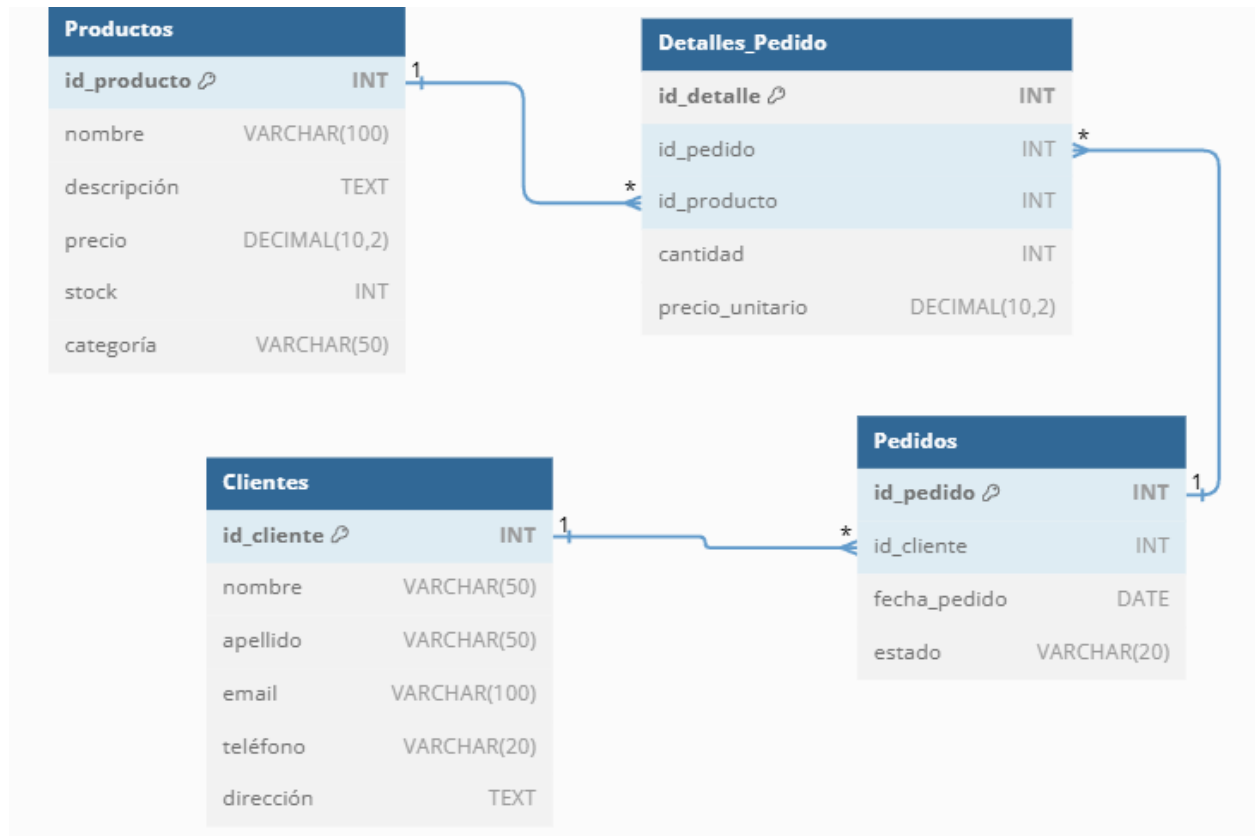
Explicación para tener en cuenta

La tienda y la bd debe ser operativa equivalente para estos ejemplos



Rubrica de evaluación para la actividad a tener en cuenta por favor leer atentamente

Diseño de la base de datos



Creación de tablas y relaciones en SQL

```
CREATE DATABASE tienda_online;
```

```
USE tienda_online;
```

```
CREATE TABLE Productos (
```

```
    id_producto INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nombre VARCHAR(100) NOT NULL,
```

```
    descripcion TEXT,
```

precio DECIMAL(10, 2) NOT NULL,

stock INT NOT NULL,

categoria VARCHAR(50)

);

CREATE TABLE Clientes (

id_cliente INT AUTO_INCREMENT PRIMARY KEY,

nombre VARCHAR(50) NOT NULL,

apellido VARCHAR(50) NOT NULL,

email VARCHAR(100) NOT NULL,

telefono VARCHAR(20),

direccion TEXT

);

CREATE TABLE Pedidos (

id_pedido INT AUTO_INCREMENT PRIMARY KEY,

id_cliente INT NOT NULL,

fecha_pedido DATE NOT NULL,

estado VARCHAR(20) NOT NULL,

FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)

);

CREATE TABLE Detalles_Pedido (

id_detalle INT AUTO_INCREMENT PRIMARY KEY,

id_pedido INT NOT NULL,

id_producto INT NOT NULL,

cantidad INT NOT NULL,

precio_unitario DECIMAL(10, 2) NOT NULL,

FOREIGN KEY (id_pedido) REFERENCES Pedidos(id_pedido),

FOREIGN KEY (id_producto) REFERENCES Productos(id_producto)

);

Relaciones entre Tablas

Las relaciones entre las tablas se establecen mediante claves foráneas (foreign keys). En el ejemplo anterior, las claves foráneas se definen en las tablas Pedidos y Detalles_Pedido

Inserción de datos

INSERT INTO Productos (nombre, descripcion, precio, stock, categoria) VALUES

('Laptop', 'Laptop de alta gama con 16GB RAM y 512GB SSD', 1200.00, 15, 'Electrónica'),

('Smartphone', 'Smartphone con pantalla de 6.5 pulgadas y 128GB de almacenamiento', 800.00,
30, 'Electrónica'),

('Auriculares', 'Auriculares inalámbricos con cancelación de ruido', 150.00, 50, 'Accesorios');

INSERT INTO Clientes (nombre, apellido, email, telefono, direccion) VALUES

('Juan', 'Pérez', 'juan.perez@example.com', '1234567890', 'Calle Falsa 123, Ciudad'),

('María', 'Gómez', 'maria.gomez@example.com', '0987654321', 'Avenida Siempre Viva 456,
Ciudad'),

('Carlos', 'López', 'carlos.lopez@example.com', '1122334455', 'Boulevard de los Sueños 789,
Ciudad');

INSERT INTO Pedidos (id_cliente, fecha_pedido, estado) VALUES

(1, '2024-09-01', 'pendiente'),

(2, '2024-09-02', 'enviado'),

(3, '2024-09-03', 'entregado');

INSERT INTO Detalles_Pedido (id_pedido, id_producto, cantidad, precio_unitario) VALUES

(1, 1, 1, 1200.00),

(1, 3, 2, 150.00),

(2, 2, 1, 800.00),

(3, 1, 1, 1200.00),

(3, 2, 1, 800.00);

Consultas SQL

Consultas Básicas

1. SELECT * FROM Productos WHERE stock > 0;

Explicación: Permite obtener una lista de todos los productos que tienen existencias disponibles, excluyendo aquellos que están agotados.

2. SELECT * FROM Clientes WHERE id_cliente = 1;

Explicación: Esto significa que obtendrás toda la información del cliente cuyo identificador es 1.

Consultas Avanzadas

3. SELECT p.nombre, SUM(dp.cantidad * dp.precio_unitario) AS total_ventas

FROM Detalles_Pedido dp

JOIN Productos p ON dp.id_producto = p.id_producto

GROUP BY p.nombre;

Explicación: proporciona una lista de productos junto con el total de ventas generadas por cada uno.

```
4. SELECT c.nombre, c.apellido, COUNT(p.id_pedido) AS numero_pedidos  
  
FROM Clientes c  
  
JOIN Pedidos p ON c.id_cliente = p.id_cliente  
  
GROUP BY c.id_cliente;
```

Explicación: proporciona una lista de clientes junto con el número de pedidos que cada uno ha realizado.

Consultas de Actualización

```
5. UPDATE Productos  
  
SET stock = stock - 1  
  
WHERE id_producto = 1;
```

Explicación: Disminuye la cantidad en inventario de un producto específico en una unidad.
Es útil, por ejemplo, cuando se realiza una venta y necesitas actualizar el inventario para reflejar la nueva cantidad disponible.

Consultas de Eliminación

```
6. UPDATE Pedidos  
  
SET estado = 'enviado'  
  
WHERE id_pedido = 1;
```

Explicación: Este tipo de comando es útil para gestionar el flujo de trabajo de los pedidos, permitiéndote actualizar el estado de los mismos a medida que avanzan en el proceso de entrega.

```
7. DELETE FROM Productos  
  
WHERE id_producto = 1;
```

Explicación: Esto significa que el producto con id_producto igual a 1 será eliminado de la base de datos.

Justificación del diseño

He diseñado la base de datos para nuestra tienda en línea siguiendo principios sólidos que aseguran su eficiencia y escalabilidad. A continuación, te explico las razones detrás de cada decisión de diseño.

Normalización y Eficiencia

Primero, he aplicado los principios de normalización para evitar la redundancia y asegurar la integridad de los datos. Cada tabla tiene un propósito claro y específico:

- **Productos:** Aquí almaceno toda la información relevante sobre los productos, como su nombre, descripción, precio, stock y categoría. Esto me permite gestionar el inventario de manera eficiente y realizar consultas rápidas sobre la disponibilidad de productos.
- **Clientes:** Esta tabla contiene los datos personales y de contacto de nuestros clientes. Es fundamental para mantener una buena relación con ellos y para gestionar sus pedidos de manera efectiva.
- **Pedidos:** En esta tabla registro cada pedido realizado por los clientes, incluyendo la fecha y el estado del pedido. Esto me ayuda a seguir el proceso de compra y entrega, asegurando que todo funcione sin problemas.
- **Detalles_Pedido:** Aquí guardo los detalles específicos de cada pedido, como los productos incluidos, la cantidad y el precio unitario. Esto me permite tener un control preciso de cada transacción y analizar las ventas de manera detallada.

Relaciones Claras y Consistentes

Las relaciones entre las tablas están claramente definidas mediante claves foráneas. Esto no solo mantiene la integridad referencial, sino que también facilita la realización de consultas complejas. Por ejemplo, puedo fácilmente obtener todos los pedidos de un cliente o los productos incluidos en un pedido específico.

Flexibilidad y Escalabilidad

El diseño de nuestra base de datos es flexible y escalable. Puedo agregar nuevos productos, clientes y pedidos sin necesidad de modificar la estructura de la base de datos. Esto es crucial para una tienda en línea que está en constante crecimiento y evolución.

Consultas Eficientes

El diseño facilita la realización de consultas eficientes para obtener información relevante. Puedo, por ejemplo, obtener todos los productos disponibles, los detalles de un pedido específico o el total de ventas por producto. Esto es esencial para la gestión operativa y la toma de decisiones estratégicas.

Integridad de los Datos

El uso de claves primarias y foráneas asegura que los datos sean únicos y que las relaciones entre las tablas se mantengan consistentes. Esto previene problemas como la duplicación de datos y la inconsistencia de la información.